

An Engineering Study of a Two-Tier QoS-Aware Scheduler for Private 5G

Should we replace the proportional-fair scheduler OpenAirInterface ships?

The decision, the measurements behind it — and the three modelling errors found on the way.

One cell, three kinds of promise

A factory floor puts traffic on one carrier whose classes want qualitatively different things:

Class	Example	What it wants	What failure looks like
GBR	Robot camera / LIDAR uplink	A sustained rate floor	Visible artefacts, lost frames
Delay	Motion control, teleoperation	Bytes before a deadline	A late command — a safety event
Best-effort	Firmware, log upload	Whatever is left	Slower; nobody notices

Proportional fair — the LTE/NR default, and what OAI ships — represents **neither a rate contract nor a deadline**. It is a good answer to a question the factory is not asking.

A GBR flow wants a specific rate or nothing of value. A delay flow wants its bytes before a deadline, after which they are worthless.

The design — and we claim no novelty for it

Tier-1 · every ~1 s

A convex program sets a target rate per flow: max-min GBR floor, then minimise contract shortfall, then maximise weighted log utility.

Tier-2 · every slot

Drift-plus-penalty virtual queues track those targets. Grants are per UE — one DCI, one transport block — filled by a MAC logical-channel multiplexer.

Configured grants

Periodic flows get standing allocations, self-gated so they stand aside when they would hurt.

Evaluated against Round Robin, Proportional Fair and a class-aware Gradient baseline. All four grant per UE, so the comparison isolates QoS-awareness rather than grant granularity.

This composes established parts — network utility maximisation over a slow horizon feeding Lyapunov drift-plus-penalty per slot.

It is the pattern behind NVS and the O-RAN RIC split. We present it because the composition, and the places it goes wrong, are instructive.

Where it wins outright (1): the control channel binds

30 dense periodic uplink sensors, 15 ms deadline. The DCI/CCE budget runs out before the data channel does.

Scheduler	On time	Worst p99
Round Robin	0/30	15.0 ms
Proportional Fair	2/30	15.0 ms
Two-tier	30/30	5.0 ms

Configured grants cost **zero PDCCH per slot and need no buffer-status round trip**. PF is throttled by both budgets at once and has no mechanism to bypass either — this is a capability gap, not a tuning gap, so no amount of PF tuning closes it.

Independently reported by Larrañaga et al. (JNCA 2023) from ns-3/5G-LENA. We reproduce it in a simulator sharing no code with theirs.

Where it wins outright (2): deadline-blindness is silent

Eight interactive 5 Mbps streams, 12 ms deadline, sharing a saturated downlink with 80 Mbps of bulk.

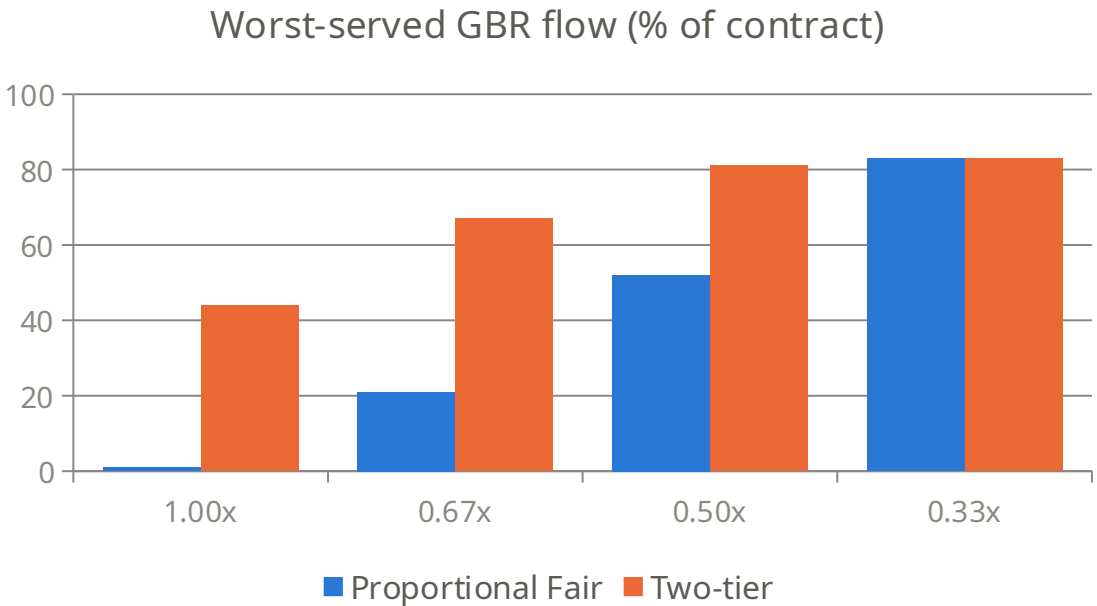
Scheduler	On time	Mean delivery	Worst p99	Bulk carried
Round Robin	3/8	84%	12.0 ms	22.8 M
Proportional Fair	5/8	86%	12.0 ms	24.6 M
Two-tier	8/8	100%	4.5 ms	13.2 M

PF's control-flow mean delivery is **86%** — **which reads healthy on a dashboard**. The missing 14% are aged-out packets: the late motion commands, the safety-relevant ones. Mean delivery is not a safety metric.

The two-tier design pays for it explicitly — bulk drops 24.6 → 13.2 Mbps to clear every deadline.

Where it is mixed: the metric differs, not the outcome

Ten robots, uplink-heavy, swept across offered load.
Two metrics disagree.



Contracts met ($\geq 95\%$ of the rate floor)

Load	PF	Two-tier
1.00x (as shipped)	1/10	0/10
0.67x	6/10	1/10
0.50x	8/10	10/10
0.33x	10/10	10/10

At 0.67x load every robot gets $\geq 67\%$ of its rate under two-tier, while PF leaves its worst at 21% — yet PF scores 6/10 to our 1/10, because six of its flows clear the 95% bar and none of ours do.

A threshold count rewards concentrating a shortfall. This design spreads it. Which is better depends on whether 90% of a video feed is worth anything — for cameras yes, for a control loop no.

Every result above is a claim about a scheduler
mediated by a simulator.

Three times, a natural-looking modelling shortcut produced a result that did not survive being corrected. One of them was a headline finding of an earlier draft of this work.

The scheduler was filling the uplink transport block

Our multiplexer was direction-agnostic — the same code split a UE's transport block in both directions, ordering flows by the **gNB's own virtual queues**. In the uplink no gNB may do that.

Mixed-flow UE (GBR video + best-effort)	RR	PF	Gradient	Two-tier
Scheduler fills the block (what we had)	3%	3%	3%	53%
UE fills the block (TS 38.321 §5.4.3.1)	100%	100%	72%	89%

We had described the top row as a clean demonstration of QoS-aware multiplexing. **It was the UE's prioritised-bit-rate configuration all along** — available to every scheduler. Correcting it also raised PF's contract count at 0.50x load from 5/10 to 8/10.

The gNB grants a block; the UE splits it by its own logical-channel prioritisation. The gNB configures the PBRs but never learns the split.

Two more, and what they have in common

Flows shared a priority level

With every flow at one priority, the multiplexer's sort silently fell back to the order flows appear in the config file. Results were reproducible only by accident.

Deriving priority from the standardised 5QI fixed it — and the numbers did not change, which is exactly why this class of error survives testing.

Tier-1 was handed the true offered load

Read straight from the traffic generator: exact, noise-free, static. No gNB has that.

Estimating it instead produced a starvation lock-in — a starved flow's data ages out, so it stops appearing in the arrival count. The flow never stopped asking; we stopped counting.

All three gave the scheduler information or authority the 3GPP split does not grant it. **All three flattered the scheduler, not the baselines** — which is not coincidence: a simulator is written from the scheduler's point of view, so its conveniences accrue there.

The check we now apply: for every quantity the scheduler reads — **which network element learns this, and how?**

So: adopt it — and do not build a fallback to PF

Two wins are unconditional

Configured grants and the deadline-aware tracker address capabilities PF lacks entirely. Both survived every fidelity correction untouched — neither scenario has a multi-flow uplink UE.

One metric loss, and it is a metric choice

Under GBR infeasibility a threshold count prefers concentration. We measured whether a knob recovers it: sweeping the max-min floor from 1 to 0 moves contracts only 1/10 $\rightarrow$ 2/10, against PF's 6/10. It does not.

And detection is not needed for safety: with the floor off, the design leads PF on mean and worst-case delivery at both loaded points while keeping both structural wins.

A scheduler-level switch to PF is the **wrong instrument** — it would surrender configured grants and deadline awareness to improve one metric on one flow class.

Only the max-min floor is regime-dependent. It is a continuous knob, and Tier-1 already computes an **exact feasibility test** — so degradation can be graceful, with no regime guessing.

What this is, and what it is not

What we offer

- Independent reproduction of two results, in a simulator sharing no code with the work that first reported them.
- Three fidelity corrections, each with the measurement it changed.
- An adoption decision with its reasoning, not just its verdict.
- Simulator, scheduler library and one script per result table.

What we do not claim

- Novelty for the design. It composes known parts.
- Silicon. This is simulation; comparative, not absolute. An OAI integration is the next step and is under way.
- External validity beyond three synthetic single-cell scenarios — no churn, no mobility, no inter-cell interference.

The remaining unmodelled effects we know of — uplink grant timing, HARQ retransmissions consuming PRBs — would **widen** the configured-grant advantage, not narrow it. Though after the last three slides, we note that is asserted from mechanism, not measured.